

AI Engineer

What You Do

You build software systems using AI at a speed and quality that wasn't previously possible. You're an architect-level thinker who uses AI as a power tool - not a junior developer with a copilot. You own architecture, code quality, infrastructure, and the effectiveness of the AI agents themselves.

You are not a traditional developer waiting for tickets. You are not isolated from the business. You are half of a two-person team that owns a project.

The Mindset

Build like it's your product. Not "the client's app" — your system. Would you deploy this to your own users?

- **Automate relentlessly.** If you're manually deploying, manually running tests, manually checking for regressions - automate it.
 - **Don't stop at "it works."** After deployment, the system needs to be observable. Error tracking. Performance monitoring. Usage analytics.
 - **Know where AI breaks.** Review AI-generated code with the same rigor you'd apply to a junior developer's PR.
 - **Understand the business problem.** Code that solves the wrong problem is worse than no code at all.
-

What You Own

Domain	What "owning it" means
Code quality	Every line meets your standard. AI writes it, you harden it - security, performance, edge cases.
Architecture	You design the system, choose the stack, define the data model.
Infrastructure	Cloud provisioning, CI/CD, monitoring, deployments. You set it up, manage it, own uptime.
Observability & resilience	Error tracking, performance monitoring, structured logging, alerting.
AI agent effectiveness	CLAUDE.md quality. Agent instructions. Intervention rate tracking.
Technical feasibility	Honest assessments when the ES proposes something. Your honesty prevents bad decisions.
Build velocity	Features ship fast and work correctly.
Security	OWASP top 10. Auth patterns. Data handling. You don't ship insecure code, even if AI generates it.

Principles

1. **Fix the instruction, not just the output.** When AI generates bad code, update CLAUDE.md so it never happens again.
 2. **Honest over optimistic.** "This will take a week" is better than "maybe 2-3 days" followed by a week.
 3. **Architecture serves the outcome, not your preferences.** The simplest one that serves the business outcome.
 4. **Ship small, ship often.** One PR, one change. Tests pass. Docs updated.
 5. **Own the full stack.** Front-end, back-end, database, infrastructure, CI/CD, observability.
 6. **Build for debuggability.** Structured logs. Error tracking. If an AI agent can't diagnose a production issue from your logs, your observability isn't good enough.
 7. **Make the ES better.** When the ES asks a technical question, teach — don't just answer.
-

Daily Rhythm

Morning (30 min): Check the control room for new requirements, decisions, or priority changes. Review overnight CI/CD results and monitoring alerts. Check Linear for current sprint priorities.

During the day: Build features using AI (Claude Code, Cursor). Review and harden AI-generated code. Architecture work. Infrastructure and observability management. Pair with the ES when they need technical input.

When the ES runs a customer call with technical content: Join for the technical portions. Provide feasibility input in real-time. Follow up on technical action items immediately after.

End of day (15 min): PR descriptions clear and linked to Linear issues. Build status updated in Linear. Blockers flagged to the ES.

Capabilities

Technical: 2-3 shipped applications end-to-end. Strong in 2-3 languages. Database design and query optimization. System architecture. Cloud infrastructure in production. CI/CD and DevOps. Security (OWASP, auth patterns, data handling).

AI-native development: Expert with Claude Code and/or Cursor. Directs AI to build complete features. Knows when to trust AI and when to rewrite. Writes effective CLAUDE.md instructions.

Communication: Explains technical decisions to non-technical people. Gives honest estimates. Clear PR descriptions and architecture docs. Raises concerns proactively.